SOLID Principles Implementation Summary

Overview

Successfully implemented SOLID principles for the Chicken Processing System by creating a domain-driven design with proper separation of concerns.

Implemented Domain Classes

1. Base Entity (`src/domain/Entity.ts`)

- **SRP**: Single responsibility for managing entity lifecycle
- **OCP**: Open for extension through inheritance
- **Features**: ID generation, timestamps, Firestore compatibility

2. Value Objects

- **Money** (`src/domain/Money.ts`): Handles currency operations
- **Address** (`src/domain/Address.ts`): Manages address data
- **UserProfile** (`src/domain/UserProfile.ts`): Manages user profile information

3. Core Domain Entities

- **User** (`src/domain/User.ts`):
  - Abstract base class following LSP
  - Concrete implementations: `CustomerUser` and `FarmStaffUser`
  - OCP: Open for new user types through inheritance
  - ISP: Segregated interfaces for different user roles
- **Product** (`src/domain/Product.ts`):
  - Abstract base class with `ProductInterface`
  - Concrete implementations: `StandardProduct`, `WholeChickenProduct`, `EggProduct`
  - LSP: All product types substitutable for base Product
- **OrderItem** (`src/domain/OrderItem.ts`):
  - Value object for order line items
  - Immutable design with update methods returning new instances
- **Order** (`src/domain/Order.ts`):
  - Manages order lifecycle and calculations
  - SRP: Handles only order-related responsibilities
  - Business logic: status transitions, total calculations

4. Enums (`src/domain/enums.ts`)

- Centralized enumeration types for type safety
- User roles, types, statuses, product types, etc.

Service Layer

UserService (`src/services/UserService.ts`)

- **SRP**: Manages only user-related operations
- **ISP**: `UserServiceInterface` with specific methods
- **DIP**: Depends on abstractions (User interface)
- **Features**: CRUD operations, filtering, business logic (approval, login tracking)

SOLID Principles Demonstrated

1. Single Responsibility Principle (SRP)

- Each class has one reason to change
- Examples:
  - `Money` handles only currency operations
  - `Address` handles only address data
  - `UserService` handles only user management

2. Open/Closed Principle (OCP)

- Classes open for extension, closed for modification
- Examples:
  - `User` class can be extended for new user types
  - `Product` hierarchy allows new product types
  - `Entity` provides base functionality for all entities

3. Liskov Substitution Principle (LSP)

- Subtypes substitutable for base types
- Examples:
  - `CustomerUser` and `FarmStaffUser` can be used as `User`
  - All product types can be used as `Product`
  - All value objects maintain expected behavior

4. Interface Segregation Principle (ISP)

- Clients shouldn't depend on interfaces they don't use
- Examples:
  - `ProductInterface` with only product-related methods
  - `UserServiceInterface` with specific user operations
  - Separate interfaces for different concerns

5. Dependency Inversion Principle (DIP)

- High-level modules depend on abstractions

- Examples:
  - `UserService` depends on `User` interface
  - Services depend on domain interfaces, not implementations
  - Domain models don't depend on infrastructure

Test Results

The test (`src/test-solid.ts`) successfully demonstrates:

1. **SRP**: Each class performs its designated responsibility
2. **OCP**: New types can be added without modifying existing code
3. **LSP**: Subtypes work correctly when substituted for base types
4. **ISP**: Interfaces are appropriately segregated
5. **DIP**: Dependencies flow toward abstractions
6. **Business Logic**: User approval, login tracking, error handling

Key Design Patterns Used

1. **Factory Pattern**: `fromFirestore` methods in entities
2. **Value Object Pattern**: Immutable objects like `Money`, `Address`
3. **Repository Pattern**: `UserService` acts as a repository
4. **Strategy Pattern**: Different user types with varying behavior
5. **Template Method Pattern**: Base `Entity` class with template methods

Benefits Achieved

1. **Maintainability**: Clear separation of concerns
2. **Testability**: Each component can be tested in isolation
3. **Extensibility**: New features can be added with minimal changes
4. **Reusability**: Domain models can be reused across services
5. **Type Safety**: TypeScript enums and interfaces prevent errors
6. **Business Logic Encapsulation**: Domain logic is in domain objects

Next Steps

1. Implement remaining services (ProductService, OrderService)
2. Add persistence layer (Firestore integration)
3. Create API controllers using the domain models
4. Add validation and error handling middleware
5. Implement unit tests for all domain classes
6. Add integration tests for services

Files Created

```
src/domain/
├── Entity.ts              # Base entity class
├── Money.ts               # Money value object
```

```
├── Address.ts             # Address value object
├── UserProfile.ts         # User profile value object
├── enums.ts               # Enumeration types
├── User.ts                # User entity hierarchy
├── Product.ts             # Product entity hierarchy
├── OrderItem.ts           # Order item value object
└── Order.ts               # Order entity

src/services/
└── UserService.ts         # User service with interface

src/test-solid.ts          # SOLID principles test
SOLID_IMPLEMENTATION_SUMMARY.md  # This document
```

The implementation successfully transforms the existing interface-based models into a proper domain-driven design following SOLID principles, providing a solid foundation for the chicken processing system.